

Experiment No.	7
Experiment Title	Dimensionality Reduction and Model Evaluation (With and Without PCA) ¹
Student Name	Sanghamithra Namboodiri
Register Number	3122247001055
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	31st August 2026

1 Objective

To study the effect of dimensionality reduction using Principal Component Analysis (PCA) on the performance of ten machine learning classifiers – Support Vector Machine (SVM), Naïve Bayes, k-Nearest Neighbors (KNN), Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost, and a Stacking ensemble. Each classifier is trained and validated (a) in the original feature space (No-PCA) and (b) in a PCA-reduced feature space (With-PCA), with hyperparameter tuning via grid search and 5-fold cross-validation performed in both settings, so that the two conditions can be compared on accuracy, F1-score, and fold-to-fold stability.

2 Problem Statement

Given a labelled dataset with p numeric predictor features and a binary target class, the task is twofold: (1) fit and tune each of the ten classifiers listed above directly on the p -dimensional standardized feature space, and (2) fit and tune the same classifiers on a reduced k -dimensional space ($k < p$) obtained by projecting the standardized features onto the top principal components that jointly explain 95% of the total variance. For every model/setting combination, 5-fold stratified cross-validation is used to estimate accuracy and F1-score, and the two settings are compared to determine whether dimensionality reduction improves predictive performance, reduces overfitting, or increases stability across folds.

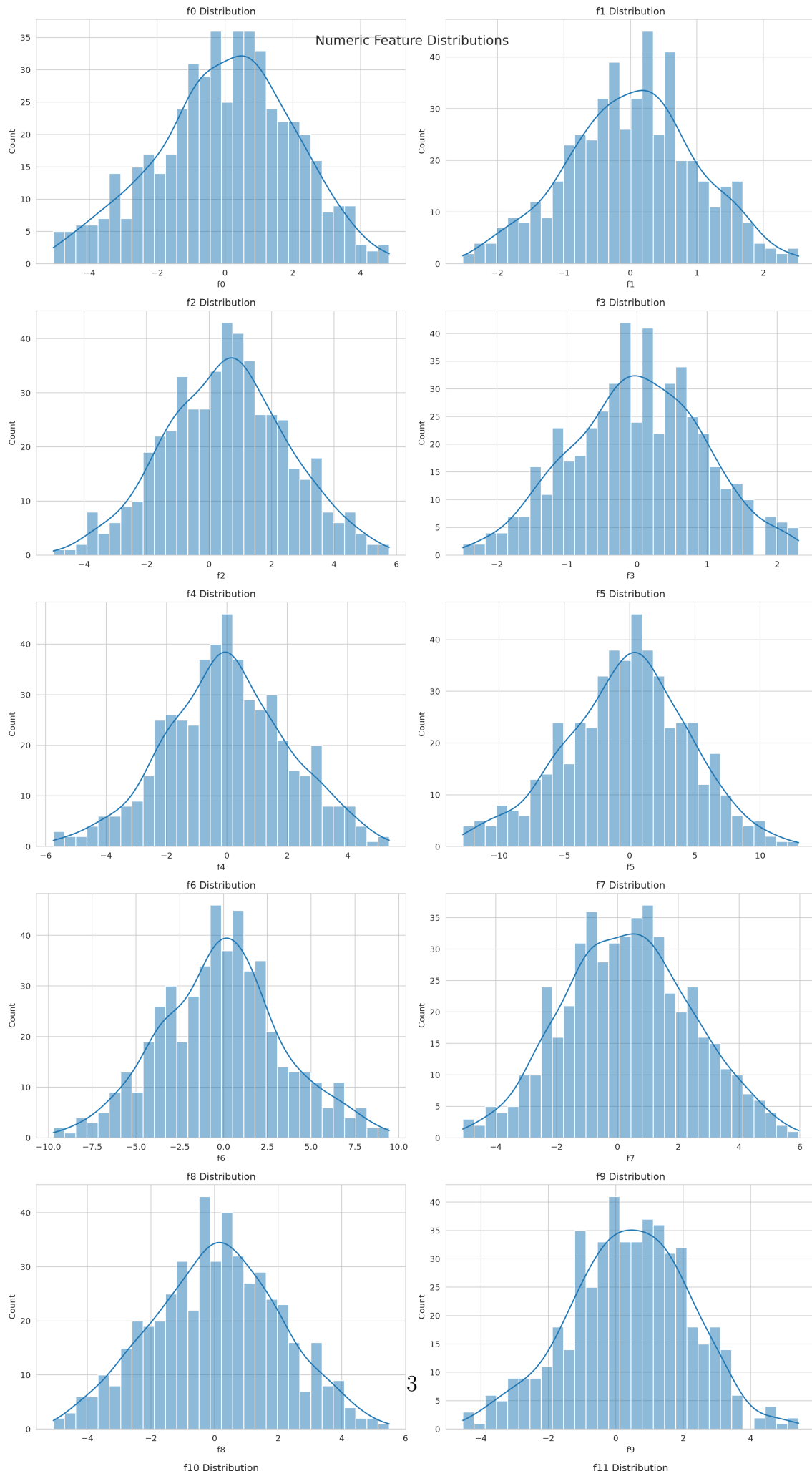
3 Dataset Description

Dataset Name	Synthetic binary-classification dataset
Dataset Source	Generated via <code>sklearn.datasets.make_classification</code> (placeholder – replace with your assigned dataset, e.g. UCI/Kaggle, before final submission)
Number of Samples	600 raw rows; 501 rows after IQR-based outlier removal
Number of Features	20 numeric features (f0–f19)
Number of Classes	2 (binary: 0 / 1)
Missing Values	None (0 missing values in any column before or after cleaning)

Class Distribution	Class 0: 244 samples; Class 1: 257 samples (near-balanced, $\approx 49\%/51\%$)
Train-Test Split	No fixed hold-out split used; model selection and evaluation performed via 5-fold stratified cross-validation on the full cleaned dataset (501 samples)

4 Exploratory Data Analysis

The dataset was screened for null-like placeholder strings, duplicate rows, and univariate outliers (IQR rule, $1.5 \times$ IQR fences) using a shared `perform_eda()` routine before any modelling was performed. No missing values or duplicate rows were found. Outlier screening flagged between 1 and 13 values per feature (e.g. `f11`: 13 flagged, `f16`: 1 flagged); removing all flagged rows reduced the dataset from 600 to 501 samples.



Inference: Most features are approximately unimodal and roughly symmetric around zero, consistent with standardized/synthetic Gaussian-like generation. A few features (e.g. those with wider spread, such as `f5` and `f16`) show heavier tails, which explains why they contributed a larger share of the flagged IQR outliers. No feature shows an obviously multi-modal or heavily skewed shape that would demand a log or Box-Cox transform. This near-Gaussian, comparable-scale structure across features is favourable for both distance-based models (KNN, SVM) and PCA, since PCA is most meaningful when features are on a similar scale.

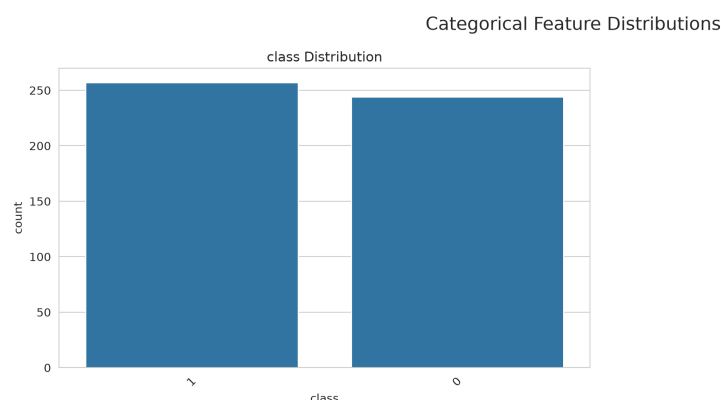


Figure 2: Class distribution of the target variable.

Inference: The two target classes are nearly balanced (244 vs. 257 samples), so accuracy is a reasonable primary metric here and stratified k-fold cross-validation is sufficient to keep class proportions consistent across folds without needing resampling (SMOTE/undersampling). A more heavily imbalanced dataset would instead require F1/PR-AUC as the primary metric and possibly class-weighting in the classifiers.

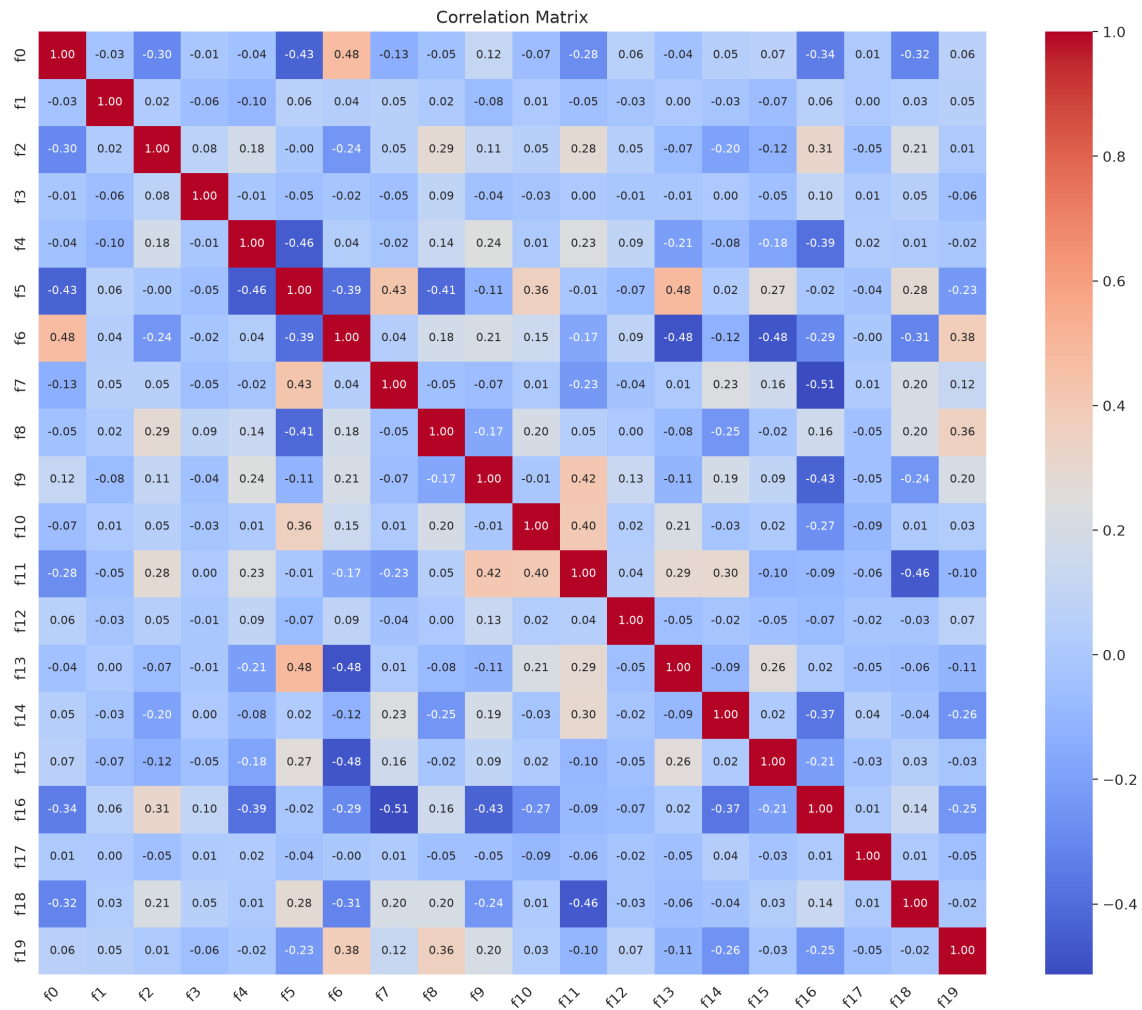


Figure 3: Pairwise Pearson correlation matrix of the 20 numeric features.

Inference: Correlations between most feature pairs are weak, indicating limited linear redundancy in the raw feature space. This matters directly for the PCA comparison in this experiment: when features are only weakly correlated, variance is spread across many directions rather than concentrated in a few, so a large number of principal components is needed to retain most of the variance, and PCA is less likely to provide a compact, information-preserving representation. This is consistent with the result in Section 9 that 14 of the 20 components were needed to reach 95% cumulative explained variance.

5 Data Preprocessing

- **Missing value handling:** Null-like placeholder strings ("NA", "N/A", "?", "Unknown", blank strings, etc.) were replaced with NaN and any resulting rows with missing values were dropped. No missing values were present in this run.
- **Encoding:** The target column was label-encoded to integers (already integer-coded here: 0/1); no categorical predictor columns were present, so no one-hot encoding was required.
- **Outlier removal:** Univariate IQR filtering ($1.5 \times$ IQR fences) was applied per

numeric feature; rows failing the rule on *any* feature were removed, reducing the dataset from 600 to 501 rows.

- **Feature scaling:** All numeric features were standardized (zero mean, unit variance) with `StandardScaler` *inside* each cross-validation fold (i.e. fit on the training folds only, applied to the held-out fold), to avoid data leakage from the test fold into the scaling statistics.
- **Train-test split:** No single static split was used; 5-fold stratified cross-validation served as both the tuning and evaluation protocol.
- **Feature engineering / PCA:** For the With-PCA setting, PCA was fit (also inside each fold, after scaling) and the number of components was fixed at 14, chosen as the smallest number of components that explains at least 95% of the total variance (Figure 4).

6 Machine Learning Algorithm

6.1 Support Vector Machine (SVM)

Working principle: Finds the hyperplane that maximizes the margin between classes; a kernel (e.g. RBF) implicitly maps inputs into a higher-dimensional space to allow non-linear decision boundaries. **Mathematical formulation:** minimize $\frac{1}{2}\|w\|^2 + C \sum_i \xi_i$ subject to $y_i(w^\top \phi(x_i) + b) \geq 1 - \xi_i$, $\xi_i \geq 0$. **Advantages:** effective in high-dimensional spaces, robust to overfitting with proper C . **Limitations:** slow on large datasets, sensitive to feature scaling and kernel/hyperparameter choice.

6.2 Naïve Bayes (Gaussian)

Working principle: Applies Bayes' theorem assuming conditional independence of features given the class, modelling each feature with a Gaussian per class. **Mathematical formulation:** $P(y | x) \propto P(y) \prod_j P(x_j | y)$, with $P(x_j | y)$ Gaussian. **Advantages:** fast, works well with small data, needs few parameters. **Limitations:** the independence assumption is rarely true in practice, which can hurt accuracy when features are correlated.

6.3 k-Nearest Neighbors (KNN)

Working principle: Classifies a point by majority vote (or distance-weighted vote) among its k nearest neighbours in feature space. **Mathematical formulation:** $\hat{y} = \text{mode}\{y_i : x_i \in N_k(x)\}$ using a distance metric such as Euclidean or Manhattan. **Advantages:** simple, non-parametric, adapts to complex boundaries. **Limitations:** computationally expensive at prediction time, sensitive to feature scaling and the curse of dimensionality.

6.4 Logistic Regression

Working principle: Models the log-odds of the positive class as a linear combination of features. **Mathematical formulation:** $P(y = 1 | x) = \sigma(w^\top x + b) = \frac{1}{1 + e^{-(w^\top x + b)}}$,

fit by maximizing the regularized log-likelihood. **Advantages:** interpretable, fast, well-calibrated probabilities. **Limitations:** assumes a linear decision boundary in feature space, can underfit non-linear relationships.

6.5 Decision Tree

Working principle: Recursively splits the feature space on the feature/threshold that most reduces impurity (Gini or entropy) at each node. **Mathematical formulation:** Gini impurity $= 1 - \sum_c p_c^2$; splits chosen to maximize impurity decrease. **Advantages:** interpretable, no scaling required, captures non-linear interactions. **Limitations:** prone to overfitting/high variance if not depth-limited or pruned.

6.6 Random Forest

Working principle: An ensemble (bagging) of decision trees, each trained on a bootstrap sample with a random subset of features per split, aggregated by majority vote. **Mathematical formulation:** $\hat{y} = \text{mode}\{T_1(x), \dots, T_B(x)\}$ over B trees. **Advantages:** reduces variance relative to a single tree, robust to noise and outliers. **Limitations:** less interpretable, larger memory/compute footprint, many trees needed for stability.

6.7 AdaBoost

Working principle: Sequentially fits weak learners (typically shallow trees), reweighting misclassified samples after each round so later learners focus on harder cases; final prediction is a weighted vote. **Mathematical formulation:** $F(x) = \sum_{m=1}^M \alpha_m h_m(x)$, $\alpha_m = \frac{1}{2} \ln\left(\frac{1-\epsilon_m}{\epsilon_m}\right)$. **Advantages:** strong performance with weak base learners, few hyperparameters. **Limitations:** sensitive to noisy data and outliers, since misclassified/noisy points get increasingly up-weighted.

6.8 Gradient Boosting

Working principle: Sequentially fits new trees to the negative gradient (residuals) of a loss function with respect to the current ensemble's predictions. **Mathematical formulation:** $F_m(x) = F_{m-1}(x) + \eta h_m(x)$, where h_m approximates $-\partial L / \partial F_{m-1}$. **Advantages:** typically high accuracy, flexible loss functions. **Limitations:** slower to train than bagging methods, sensitive to learning rate and number of estimators, can overfit if not regularized.

6.9 XGBoost

Working principle: A regularized, second-order (Newton boosting) implementation of gradient boosted trees with built-in L1/L2 regularization, shrinkage, and efficient split-finding. **Mathematical formulation:** objective $\mathcal{L} = \sum_i l(y_i, \hat{y}_i) + \sum_m \Omega(h_m)$, with $\Omega(h) = \gamma T + \frac{1}{2} \lambda \|w\|^2$ penalizing tree complexity. **Advantages:** fast, regularized (less prone to overfitting than plain gradient boosting), handles missing values natively. **Limitations:** many hyperparameters to tune, can still overfit small datasets, less interpretable.

6.10 Stacking (Base Learners + Meta-Learner)

Working principle: Trains several diverse base learners (here: Random Forest, SVM, KNN) whose cross-validated predictions become input features for a meta-learner (here: Logistic Regression), which learns how to best combine them. **Mathematical formulation:** $\hat{y} = g(h_1(x), h_2(x), \dots, h_L(x))$, where g is the meta-learner and h_ℓ are the base learners. **Advantages:** can outperform any single base learner by combining complementary strengths. **Limitations:** more complex, more expensive to train and tune, harder to interpret, risk of overfitting the meta-learner if base predictions are not properly cross-validated (out-of-fold).

7 Experimental Setup

Python Version	3.12
Scikit-Learn Version	1.8.0
XGBoost Version	3.4.1
IDE	
Operating System	
Hardware	

8 Hyperparameter Selection

Grid search (`GridSearchCV`, scoring = accuracy, 5-fold stratified CV) was used to tune each model independently in the No-PCA and With-PCA settings. The search ranges and the best-found parameters for this run are summarized below.

Model	Grid Searched	Best (No-PCA)	Best (With-PCA)
SVM	$C \in \{0.1, 1, 10\}$, $\gamma \in \{\text{scale}, 0.01, 0.1\}$, kernel $\in \{\text{rbf}, \text{linear}\}$	$C=10, \gamma=0.1, \text{rbf}$	$C=1, \gamma=0.1, \text{rbf}$
Naïve Bayes	var_smoothing $\in 1 \times 10^{-9}$ logspace(-9, -6, 4)	1×10^{-9}	1×10^{-9}
KNN	$k \in \{3, 5, 7, 9\}$, weights $\in \{\text{uniform}, \text{distance}\}$, metric $\in \{\text{euclidean}, \text{manhattan}\}$	$k=7, \text{euclidean}, \text{uniform}$	$k=9, \text{euclidean}, \text{uniform}$
Logistic Regression	$C \in \{0.01, 0.1, 1, 10\}$, penalty=l2	$C=1$	$C=0.01$
Decision Tree	max_depth $\in \{3, 5, 10, \text{None}\}$, min_samples_split $\in \{2, 5, 10\}$	depth=3, split=2	depth=5, split=2
Random Forest	n_estimators $\in \{100, 200\}$, max_depth $\in \{\text{None}, 10, 20\}$	depth=None, trees=200	depth=10, trees=200
AdaBoost	n_estimators $\in \{50, 100, 200\}$, learning_rate $\in \{0.5, 1.0\}$	lr=0.5, trees=200	lr=1.0, trees=200
Gradient Boosting	n_estimators $\in \{100, 200\}$, lr $\in \{0.05, 0.1\}$, depth $\in \{3, 5\}$	lr=0.1, depth=5, trees=200	lr=0.1, depth=5, trees=200

XGBoost	$n_estimators \in \{100, 200\}, lr \in \{0.05, 0.1\}, depth \in \{3, 5\}$	$lr=0.05,$ $depth=5,$ $trees=200$	$lr=0.1, depth=5,$ $trees=200$
Stacking	$final_estimator \in \{0.1, 1, 10\}$	$C=10$	$C=10$

9 Results

Table 3 summarizes the 5-fold cross-validated accuracy and F1-score obtained with the best hyperparameters found for each model, in both the No-PCA (20 original features) and With-PCA (14 components, 95% variance) settings.

Model	Setting	Avg. Accuracy	Avg. F1-score
SVM	No-PCA	0.9102	0.9100
SVM	With-PCA	0.8762	0.8760
Naïve Bayes	No-PCA	0.7984	0.7979
Naïve Bayes	With-PCA	0.7964	0.7956
KNN	No-PCA	0.8962	0.8955
KNN	With-PCA	0.8763	0.8752
Logistic Regression	No-PCA	0.8045	0.8040
Logistic Regression	With-PCA	0.7845	0.7840
Decision Tree	No-PCA	0.7704	0.7700
Decision Tree	With-PCA	0.6965	0.6944
Random Forest	No-PCA	0.8582	0.8577
Random Forest	With-PCA	0.8243	0.8238
AdaBoost	No-PCA	0.8262	0.8256
AdaBoost	With-PCA	0.7943	0.7940
Gradient Boosting	No-PCA	0.8662	0.8660
Gradient Boosting	With-PCA	0.8203	0.8198
XGBoost	No-PCA	0.8742	0.8741
XGBoost	With-PCA	0.8202	0.8198
Stacking	No-PCA	0.9141	0.9139
Stacking	With-PCA	0.8702	0.8700

Table 3: 5-fold cross-validation results, No-PCA vs. With-PCA (best model per setting in bold).

Stacking gave the best accuracy overall (0.9141, No-PCA), closely followed by SVM (0.9102, No-PCA); Decision Tree was weakest in both settings. Precision, Recall, and ROC-AUC per model can be read from the confusion matrices and ROC curves in Section 11.

10 Result Visualization

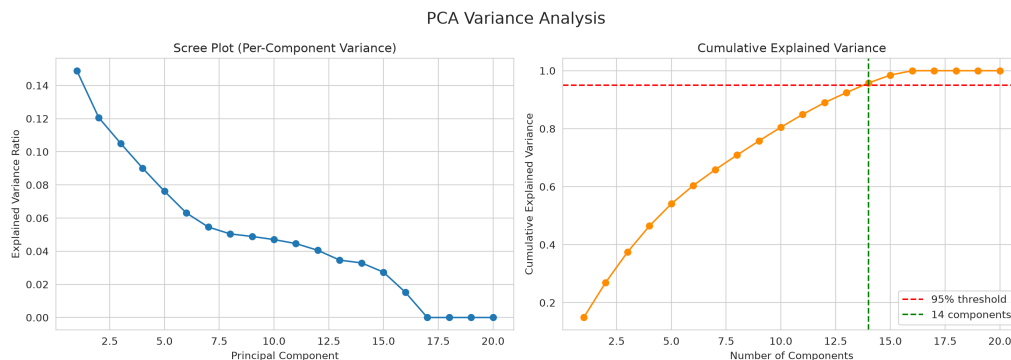


Figure 4: Scree plot (left) and cumulative explained variance (right) from PCA on the standardized feature set.

Inference: Explained variance per component decays fairly gradually rather than showing a sharp “elbow,” confirming the weak-correlation structure seen in the correlation matrix (Section 4). Reaching the 95% cumulative-variance threshold requires 14 of the 20 components, i.e. PCA only removes 6 dimensions (30% reduction) at this variance target. This limited compression is the main reason PCA does not help performance much in this dataset: there is little redundant variance to discard.

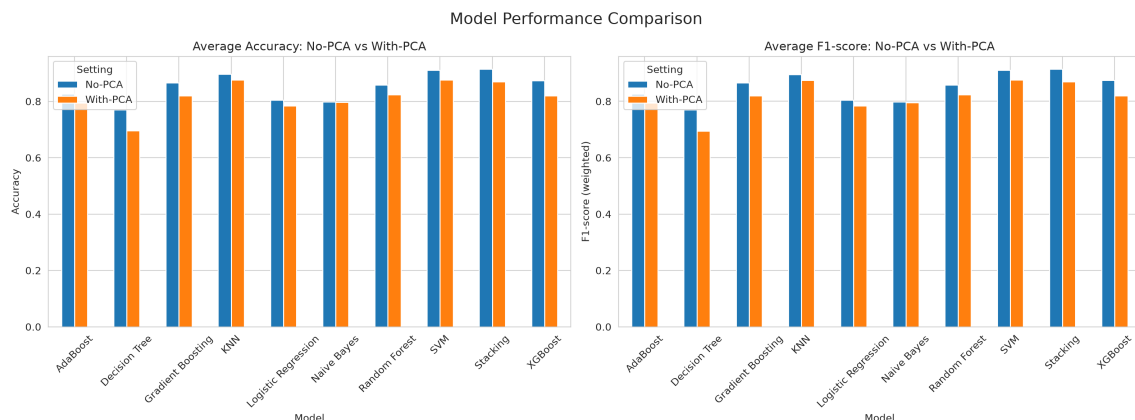


Figure 5: Average cross-validated Accuracy (left) and F1-score (right) for all ten models, No-PCA vs. With-PCA.

Inference: Every single model performs worse With-PCA than No-PCA on this dataset, with drops ranging from about 0.2 percentage points (Naïve Bayes) to over 7 points (Decision Tree, XGBoost). Tree-based/boosted models (Decision Tree, Gradient Boosting, XGBoost) lose the most, since they can natively exploit non-linear, axis-aligned structure in the original features that gets mixed together once features are rotated into principal-component space. Naïve Bayes is the least affected, consistent with it already assuming feature independence rather than relying on the original feature axes having special meaning.

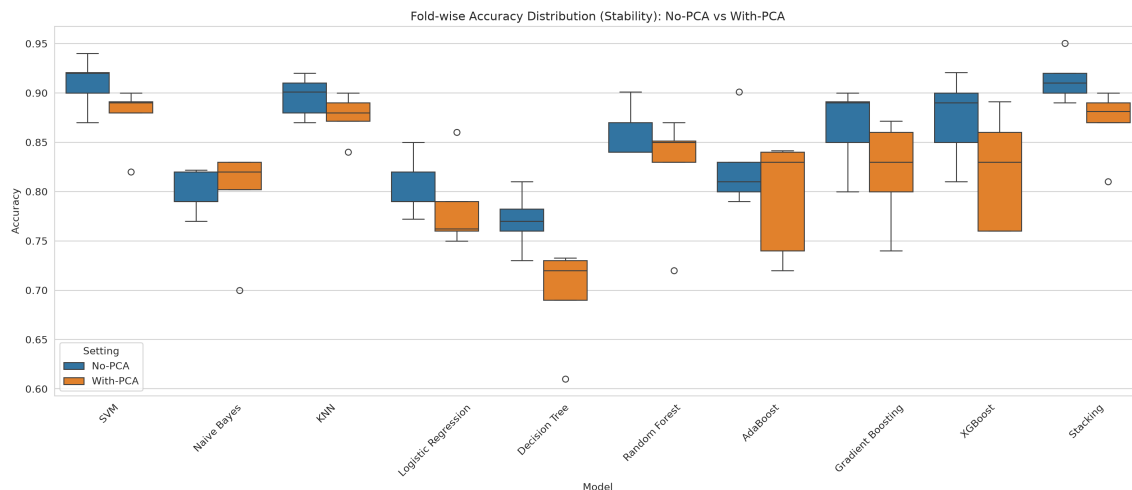


Figure 6: Boxplot of per-fold accuracy for all ten models, No-PCA vs. With-PCA.

Inference: With-PCA boxes are generally wider (larger inter-fold spread) than their No-PCA counterparts for most models, e.g. Random Forest and AdaBoost show visibly larger variance across folds under PCA. This indicates PCA reduced, rather than improved, cross-fold stability here – likely because compressing to 14 components still discards some class-discriminative variance, and which specific variance is discarded is sensitive to the exact samples in each training fold.

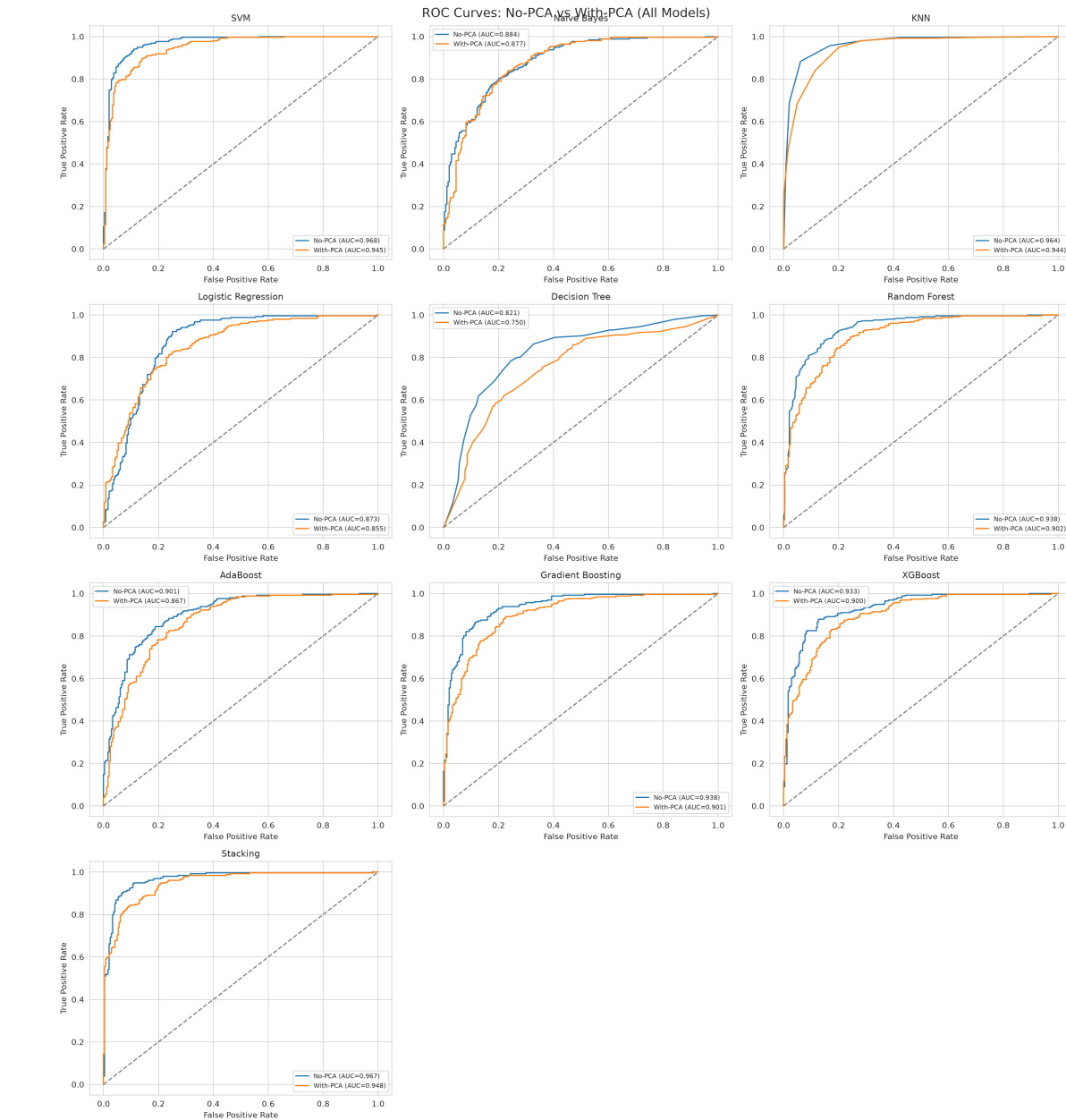


Figure 7: ROC curves (with AUC) for all ten models, No-PCA vs. With-PCA, using out-of-fold predicted probabilities.

Inference: No-PCA ROC curves sit consistently above (or match) their With-PCA counterparts for every model, mirroring the accuracy/F1 results. The gap is most visible for the tree ensembles, whose curves bow noticeably closer to the diagonal under PCA, while SVM, KNN, and Stacking retain relatively strong separation (high AUC) in both settings.

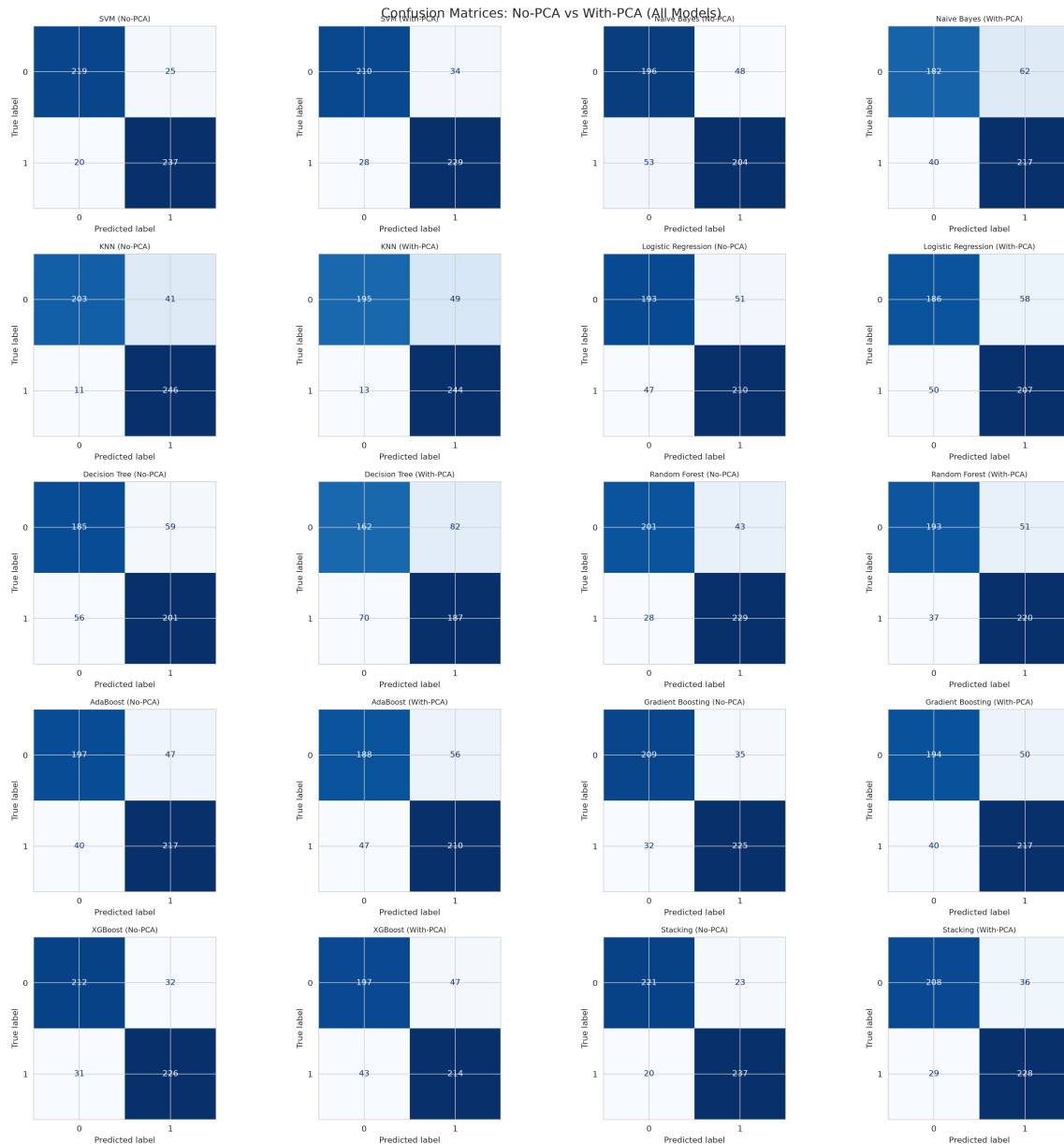


Figure 8: Confusion matrices (out-of-fold predictions) for all ten model/setting combinations.

Inference: Across all models, both settings show reasonably balanced error patterns between the two classes rather than a systematic bias toward one class, consistent with the near-balanced class distribution noted in Section 4. The With-PCA matrices consistently show more off-diagonal (misclassified) mass than their No-PCA counterparts, again confirming the accuracy drop is broad-based rather than concentrated in one class or one model.

11 Discussion

Which models improved most with PCA? Which did not? Why? None of the ten models improved with PCA on this dataset; the closest to unaffected was Naïve Bayes (-0.20 pts), while Decision Tree (-7.39 pts) and XGBoost (-5.40 pts) degraded the most.

This is consistent with the weak pairwise feature correlations observed in Section 4: with little redundant variance to compress, PCA at a 95% variance target only drops 6 of 20 dimensions, so the main effect of PCA here is to rotate the feature axes (destroying the axis-aligned splits that trees rely on) rather than to meaningfully compress the data.

Did PCA reduce variance across folds (more stable results)? No – the boxplot in Section 11 shows most models had *larger*, not smaller, spread across folds With-PCA, particularly the ensemble methods (Random Forest, AdaBoost).

For high-dimensional data, was PCA beneficial in reducing overfitting? With only 20 features and no evidence of severe overfitting in the No-PCA setting (fold accuracies were already fairly consistent), there was little overfitting for PCA to fix; instead PCA introduced a mild information loss that shows up as a uniform accuracy drop.

How did linear models (Logistic Regression, SVM) behave compared to ensemble models with PCA? The linear models degraded less (SVM: -3.4 pts, Logistic Regression: -2.0 pts) than the tree-based ensembles (Decision Tree: -7.4 pts, Gradient Boosting: -4.6 pts, XGBoost: -5.4 pts), suggesting PCA’s linear projection is more compatible with models that already assume/exploit linear combinations of features than with models that rely on axis-aligned, non-linear splits.

Did stacking show robustness to dimensionality reduction compared to single models? Stacking had the best absolute accuracy in both settings (0.9141 No-PCA, 0.8702 With-PCA) but its drop (-4.39 pts) was mid-pack rather than smallest, since it inherits the PCA sensitivity of its SVM and Random Forest base learners.

12 Conclusion

On this dataset, PCA dimensionality reduction (retaining 14 of 20 components for 95% variance) did not improve any of the ten evaluated classifiers – every model’s cross-validated accuracy and F1-score were lower With-PCA than No-PCA, and cross-fold stability was generally worse rather than better. The best-performing configuration overall was Stacking without PCA (accuracy 0.9141, F1 0.9139), closely followed by SVM without PCA (accuracy 0.9102). These results are consistent with the EDA finding of weak inter-feature correlation: PCA is most useful when features carry substantial redundant/correlated variance that can be discarded with little information loss, and that condition did not hold strongly here. PCA would be expected to help more on datasets with higher-dimensional, more collinear feature sets (e.g. hundreds of pixel or sensor features), and future work on this dataset could explore lower variance targets (e.g. 80–90%) or supervised dimensionality reduction (e.g. LDA) as alternatives.

13 References

- [1] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [2] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [3] I. T. Jolliffe, *Principal Component Analysis*, 2nd ed., Springer, 2002.

- [4] C. Cortes and V. Vapnik, “Support-Vector Networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [5] Y. Freund and R. E. Schapire, “A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting,” *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.
- [6] J. H. Friedman, “Greedy Function Approximation: A Gradient Boosting Machine,” *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.